

Lista de Exercícios Práticos 1

Desenvolvimento Web: HTML5, CSS3, Flexbox e JavaScript Moderno

Tema Integrador: Plataforma de Adoção de Pets (“Adote um Amigo”)

Prof.: Thiago Soares Marques

Questão 01: Estrutura, Semântica e Acessibilidade

Crie um arquivo chamado `index.html` e construa a estrutura de uma página para a plataforma de adoção de animais utilizando **apenas** tags semânticas do HTML5. A página deve conter obrigatoriamente os seguintes blocos estruturais:

- a) **Cabeçalho (<header>):** Deve conter o logotipo ou título do projeto (“Adote um Amigo”) e a navegação principal (<nav>). A navegação deve ser construída utilizando uma lista não ordenada (e) com os links: Início, Animais e Como Adotar.
- b) **Área Principal (<main>):**
 - **Seção de Destaque (<section>):** Um banner de boas-vindas contendo um título principal (<h1>) e um parágrafo explicativo sobre a missão do abrigo.
 - **Vitrine de Pets (<section>):** Deve conter pelo menos três “cards” de animais disponíveis. Cada animal deve ser englobado individualmente na tag <article>.
 - Dentro de cada card (<article>), utilize a tag <figure> para a foto do animal e <figcaption> para o nome dele. O atributo alt nas imagens é **obrigatório** para garantir a acessibilidade.
 - Utilize a tag <time> para indicar explicitamente a data de resgate ou de nascimento de cada pet.
- c) **Barra Lateral (<aside>):**
 - **Horários:** Uma tabela (<table>) estruturada corretamente com as tags <thead>, <tbody>, <tr>, <th> e <td>, exibindo os dias da semana e os respectivos horários de funcionamento para visitação do abrigo.
 - **Links Úteis:** Logo abaixo da tabela de horários, adicione uma nova seção de navegação (<nav>) contendo uma lista de links externos relevantes (ex: “Como denunciar maus-tratos”, “Guia de vacinação animal”, “Blog do Abrigo”).
- d) **Rodapé (<footer>):**
 - Inclua as informações de contato do abrigo estruturadas dentro da tag <address>.
 - Adicione uma seção de “Perguntas Frequentes” utilizando as tags interativas nativas <details> e <summary>.

Observação: O uso da tag genérica <div> é **estritamente proibido** nesta questão. Toda a hierarquia e organização do documento deve ser resolvida por meio de tags semânticas apropriadas.

Questão 02: Layout e Box Model

Crie um arquivo chamado `style.css`, vincule-o ao seu documento HTML e estilize a interface aplicando os seguintes requisitos técnicos obrigatórios:

- a) **Reset e Box Model:** Aplique um reset básico inicial limpando as margens (`margin: 0`) e preenchimentos (`padding: 0`) utilizando o seletor universal (*). Configure explicitamente a propriedade `box-sizing: border-box`. Defina uma fonte base limpa (ex: *Arial*, *Helvetica*, *sans-serif*) e escolha uma paleta de cores consistente (uma cor primária para destaques, uma cor escura para textos e tons claros para fundos).

b) Layout Macro com Flexbox:

- O `<header>` deve atuar como um *Flex Container*, alinhando o logotipo à esquerda e a `<nav>` à direita (`justify-content: space-between`), garantindo também o alinhamento vertical centralizado.
- O corpo principal da página (contendo o `<main>` e o `<aside>`) deve ser disposto lado a lado usando Flexbox. Atribua proporcionalidade ao layout utilizando propriedades flex (ex: `flex: 3` para a área principal e `flex: 1` para a barra lateral), adicionando um espaçamento (`gap`) confortável entre eles.

c) Layout Micro (Cards e Links):

- A seção da vitrine de pets deve ser configurada como um *Flex Container* com suporte a quebra de linhas (`flex-wrap: wrap`) e alinhamento responsivo.
- Cada `<article>` (card do pet) deve possuir uma largura base inicial (ex: `flex-basis: 300px`), bordas arredondadas (`border-radius`), cor de fundo clara distinta do fundo da página e uma sombra suave (`box-shadow`).
- Os itens da lista de “Links Úteis” localizados no `<aside>` devem ser estilizados removendo o sublinhado padrão dos links (`text-decoration: none`) e aplicando um espaçamento vertical adequado entre eles.

d) Interatividade Visual e Pseudo-classes:

- Ao passar o ponteiro do mouse (`:hover`) sobre os cards dos pets, eles devem sofrer uma leve elevação visual através da propriedade `transform: translateY()` e um aumento sutil da sombra. Configure a propriedade `transition` para suavizar esse efeito.
- Estilize as linhas da tabela de horários no `<aside>` utilizando as pseudo-classes `:nth-child(even)` ou `:nth-child(odd)` para criar um efeito zebrado harmônico.

Questão 03: Validação de Formulário com Eventos

Adicione um formulário com o título “Demonstre Interesse” na barra lateral (`<aside>`), posicionado logo abaixo da seção de links úteis. O formulário deve conter um campo de texto para o **Nome**, um campo para o **E-mail** e um botão de envio. Em um arquivo separado chamado `script.js`, implemente a seguinte lógica comportamental:

1. Capture o evento de submissão (`submit`) do formulário através do JavaScript.
2. Previna imediatamente o comportamento padrão de recarregamento da página (`preventDefault()`).
3. Realize a validação dos campos de texto verificando se ambos foram preenchidos corretamente (não estão vazios ou compostos apenas por espaços).
4. **Fluxo de Erro:** Caso algum campo esteja em branco, mude a cor da borda do respectivo `input` para vermelho e injete dinamicamente um parágrafo de erro (`<p>`) logo abaixo do formulário com o aviso “Preencha todos os campos”.
5. **Fluxo de Sucesso:** Se ambos os dados forem válidos, exiba um `alert` na tela contendo a mensagem “Interesse registrado com sucesso!” e limpe por completo todos os campos do formulário.

Questão 04: Destructuring e Template Literals

Considere o seguinte objeto JavaScript que simula o retorno estruturado de um banco de dados relacional contendo informações sobre um pet do abrigo:

```
1 const pet = {  
2   id: 404,  
3   nome: "Bidu",  
4   especie: "Cachorro",  
5   abrigo: {  
6     nome: "Amigo Fiel",  
7     cidade: "Campina Grande"  
8   },  
9   status: "Disponível para adoção"  
10 };
```

Escreva uma *arrow function* chamada `exibirFichaPet`. A função deve receber o objeto acima como parâmetro de entrada, utilizar a técnica de **destructuring** para extrair diretamente as propriedades `nome`, `especie`, `cidade` (aninhada dentro de `abrigo`) e `status`. Por fim, a função deve retornar uma string estritamente formatada utilizando **template literals** para gerar a seguinte saída:

“O Cachorro Bidu está Disponível para adoção no abrigo da cidade de Campina Grande.”

Questão 05: Programação Funcional (Map, Filter e Reduce)

O departamento financeiro do abrigo armazena o histórico das despesas operacionais mensais dedicadas aos cuidados dos animais utilizando um array contendo múltiplos objetos:

```
1 const despesas = [  
2   { item: "Ração Seca 15kg", valor: 140, categoria: "Alimentação" },  
3   { item: "Vacina V10", valor: 85, categoria: "Saúde" },  
4   { item: "Vermífugo", valor: 40, categoria: "Saúde" },  
5   { item: "Sachês de Carne", valor: 35, categoria: "Alimentação" },  
6   { item: "Antipulgas", valor: 70, categoria: "Saúde" }  
7 ];
```

Utilizando **exclusivamente** os métodos funcionais de array `filter()`, `map()` e `reduce()`, elabore um bloco de código encadeado (*method chaining*) capaz de realizar sequencialmente as seguintes operações aritméticas e lógicas:

1. **Filtrar:** Isole apenas os objetos cuja propriedade `categoria` seja estritamente igual a “Saúde”.
2. **Mapear:** Aplicando um acordo de parceria firmado com um laboratório veterinário local, conceda um desconto linear de 15% sobre o valor original de cada um dos itens de saúde previamente filtrados (multiplicando o valor por 0.85).
3. **Reduzir:** Calcule de forma acumulada o valor total consolidado a ser despendido com a saúde dos animais após a aplicação de todos os descontos individuais.

Imprima o valor final obtido diretamente no console do navegador.